

 **OntarioTech**  
Engineering  
& Applied Science



CANADIAN ARTIFICIAL INTELLIGENCE ASSOCIATION  
ASSOCIATION POUR L'INTELLIGENCE ARTIFICIELLE AU CANADA

**Canadian AI**  **2025**  
26–29 May 2025, Calgary, Alberta

# Agile Reinforcement Learning for Real-Time Task Scheduling in Edge Computing

Authors: Amin Avan<sup>(presenter)</sup>, Akramul Azim, Qusay Mahmoud.



## Introduction:

- Real-time applications
  - Soft real-time applications (SRTA)
    - significant processing demands.
    - execution on embedded devices.
    - offloading portions of workload to edge computing (EC).
  - Hard real-time applications
- Task scheduling
  - Performance of SRTA depends on timeliness & accuracy of 'task schedules'.
  - Task scheduling in EC is NP-hard
    - optimizing multiple objectives for both edge users & edge servers
    - evaluating numerous parameters inherent in EC
    - managing the dynamic EC environment
  - Heuristic- & meta-heuristic-based task scheduling algorithm struggle to find optimal/near-optimal 'task schedules', while RL is capable.



## Introduction:(cont)

- Edge computing (EC)
  - Edge users can utilize idle edge server resources.
  - Edge servers: routers, switches, single-board computers, system-on-chip devices, server-grade machines.
    - Edge servers located in base stations, access points, roadside units
  - Geographically dispersed & distributed
  - Heterogeneous computational, RAM, storage
  - Lower & reliable communication latency
- Reinforcement learning (RL)
  - Learns an optimal policy by interacting w/ environment via actions & receiving rewards.
  - Models complex environments by encoding multiple factors into states.
  - Dynamically updates its policies as conditions change.



## Main Related works:

- Deep Post-Decision States (DPDS) [9]:
  - A Deep Reinforcement Learning task scheduling algorithm
    - DPDS = Post-Decision States + Deep Deterministic Policy Gradient.
  - To accelerate RL-based task scheduling algorithm
    - Age-of-Information is incorporated to prioritize fresh environmental data.
    - Task scheduling problem is formulated as Constrained Markov Decision Process (CMDP)
  - Improves 'power consumption' & 'AoI'
  - Neglects the real-time requirements of tasks.
  - Assumes the presence of a single edge server.

## Main Related works:(Cont.)

- Decentralized Task Offloading Scheduling Algorithm (DOSA) [10]:
  - a Deep Reinforcement Learning task scheduling algorithm
    - employs Double-DQN, Dueling-DQN, Prioritized Replay Memory, & Recurrent Neural Network
  - Considers `communication costs' & `operational status' of edge servers to reduce latency.
  - Accounts for required processor cycles & data transmission sizes of tasks
  - Overlooks heterogeneity of edge servers, remaining processing capacity, memory volume, response time

## Main Related works:(Cont.)

- distributed computation (DCOM) [11]:
  - A Deep Reinforcement learning based approach
  - Designed to
    - maximize edge user QoS
    - minimize latency & power usage
  - Encounters scalability challenges
    - Large state-action space & DAG-based task dependencies significantly prolong RL-agent learning time.
    - RL struggles w/ normal- & large-scale problems.



## Problem statement:

- Most existing schedulers are designed for general-purpose applications, while SRTAs require metrics as hit-ratio, deadline, response time [4, 8–19].
- Overlooks runtime of task scheduling algorithms.
- RL-based approaches suffer from prolonged learning time
- Prolonged learning time stems from trial-&-error in a large state-action space [9, 20].



## Problem formulation:

- aRL's goal: Maximize hit-ratio w/ minimal runtime
  - maximize hit-ratio:
    - C1.1: Each task is assigned to an edge server only if its response time meets its deadline.
    - C1.2: Task is allocated only if the edge server has sufficient resources.
  - minimize RL agent learning time (runtime):
    - C2.1: Each task is assigned to one edge server only.
    - C2.2: An edge server is available if all resource utilizations stay below a threshold.

## Agile Reinforcement Learning (aRL):

- We formulate task scheduling problem w/ MDP.
- Decision matrix(G) records task-to-server assignments by aRL.
- Action masking & informed exploration accelerate aRL's learning.
  - Action masking
    - Actions per episode are capped by task count.
    - Assignments per task are capped at one.
  - Informed exploration
    - Explores the state-action space using an EDF.

$$\mathbf{G} = \begin{matrix} & s_1 & \dots & s_m \\ \tau_1 & \begin{bmatrix} x_{11} & \dots & x_{1m} \\ \vdots & \vdots & \vdots \\ x_{n1} & \dots & x_{nm} \end{bmatrix} \end{matrix}$$

# Agile Reinforcement Learning (aRL):(Cont.)

- aRL-agent gets P-reward per satisfied constraint & N-reward per violation.
- Reward Function
  - P-reward per satisfied constraint
    - reward is multiplied by task criticality
  - N-reward per violated constraint

**Input:** Set of  $p$  offloaded tasks from edge users  $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_p\}$  where  $(\tau_j \in \Gamma_{\delta_i}, \forall \delta_i \in \mathcal{E})$   
 Set of  $q$  edge servers  $\mathcal{S} = \{s_1, s_2, \dots, s_q\}$ ,  $NumActionsTaken = 0$

**Output:** Generated task schedule ( $\Phi$ )

```

1: function GetEDFtask():
2:    $\Gamma_{ua} \leftarrow \text{Sort}(\Gamma_{ua})$  by  $d_{\tau_j}$  in ascending order; return  $\Gamma_{ua}[0]$ 
3: function SelectAction(state):
4:   Compute  $\varepsilon_{\text{threshold}}$ ; Compute  $sample \leftarrow \text{Random}(0, 1)$ ; Update( $\Gamma_{ua}$ ) based on Eq. (4.7);
5:   if ( $sample > \varepsilon_{\text{threshold}}$ ) then
6:      $actionValues \leftarrow \text{policyNet}(state)$ ;  $bestAction \leftarrow \arg \max(actionValues)$ ;
7:     Return  $\text{mapAction}(bestAction)$ 
8:   else Return  $\text{mapAction}(\text{GetEDFtask}())$ 
9: begin main
10: for episode in range(episodes) do
11:    $G \leftarrow \text{matrix}(0_{|\mathcal{T}| \times |\mathcal{S}|})$ ;  $t, reward, hitTasks \leftarrow 0, 0, 0$ ;  $\Phi \leftarrow []$ ;
12:   while ( $t \leq |\mathcal{T}|$ ) or ( $hitTasks == |\mathcal{T}|$ ) do ## 'action bound' (Eq.(4.4))
13:      $\Gamma_{ua} = \{\tau_j \in \mathcal{T} : \sum_{k=1}^{|\mathcal{S}|} G_{j,k} = 0\}$ ;
14:      $a_t = \text{SelectAction}(state)$ ; ##  $a_t : (\tau_j, s_k)$ 
15:      $t++$ ;
16:     if [ $(a_t(\tau_j) \in \Gamma_{ua}) == \text{TRUE}$ ] then ## 'single-assignment constraint' (Eq.(4.7))
17:        $reward++positiveReward$ ;
18:       if ( $(U_{Ps_k, Ms_k, Ls_k} < U_{th})$  and ( $(U_{Ps_k, Ms_k, Ls_k} + u_{p\tau_j, m\tau_j, l\tau_j} < 1)$ ) then ## edge server capacity
19:         constraints (Eqs.(3.17),(3.11))
20:          $reward++positiveReward$ ;
21:         if ( $(r_{\tau_j:s_k} \leq d_{\tau_j})$ ) then ## response time against deadline (Eq.(3.10))
22:            $reward++positiveReward$ ;  $hitTasks++1$ ;
23:         else  $reward++negativeReward$ ;
24:       else  $reward++negativeReward$ ;
25:        $\Phi[t] \leftarrow a_t$ ; ## append (task  $\tau_j$ , edge server  $s_k$ ) pair to the task schedule
26:     end while
27:   end for
28: end main
    
```

Fig. 1. Agile Reinforcement Learning (aRL) Algorithm

# Experimental results:

- EC: 22 zones, each w/ a base station & network switch.
- Video surveillance app is evaluated in EC.
  - Edge users: resource-constrained cameras
    - 52 edge users are randomly placed.
    - Services: 44×Crowd Counting, 6×Face Recognition, 1 ML Model per type.
      - Each service consists of tasks w/ distinct requirements.
  - Edge servers
    - 4 edge servers deployed at separate BSs
    - Types: NVIDIA Jetson TX2, Intel Xeon E5430, Intel Xeon E5645
- Algorithms evaluated at hit-ratio by
  - Runtime
  - RAM usage
  - power consumption

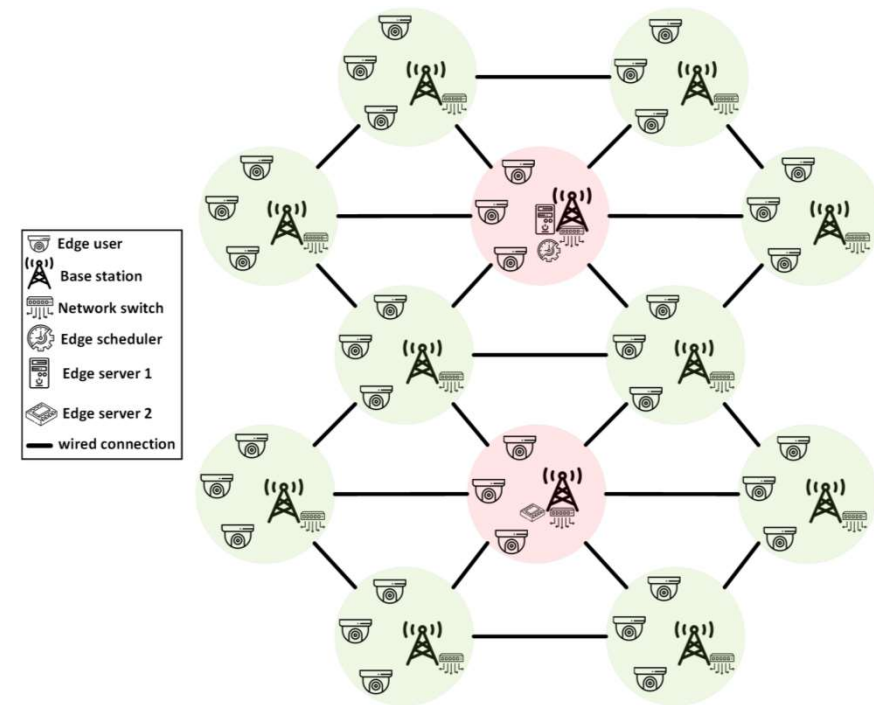
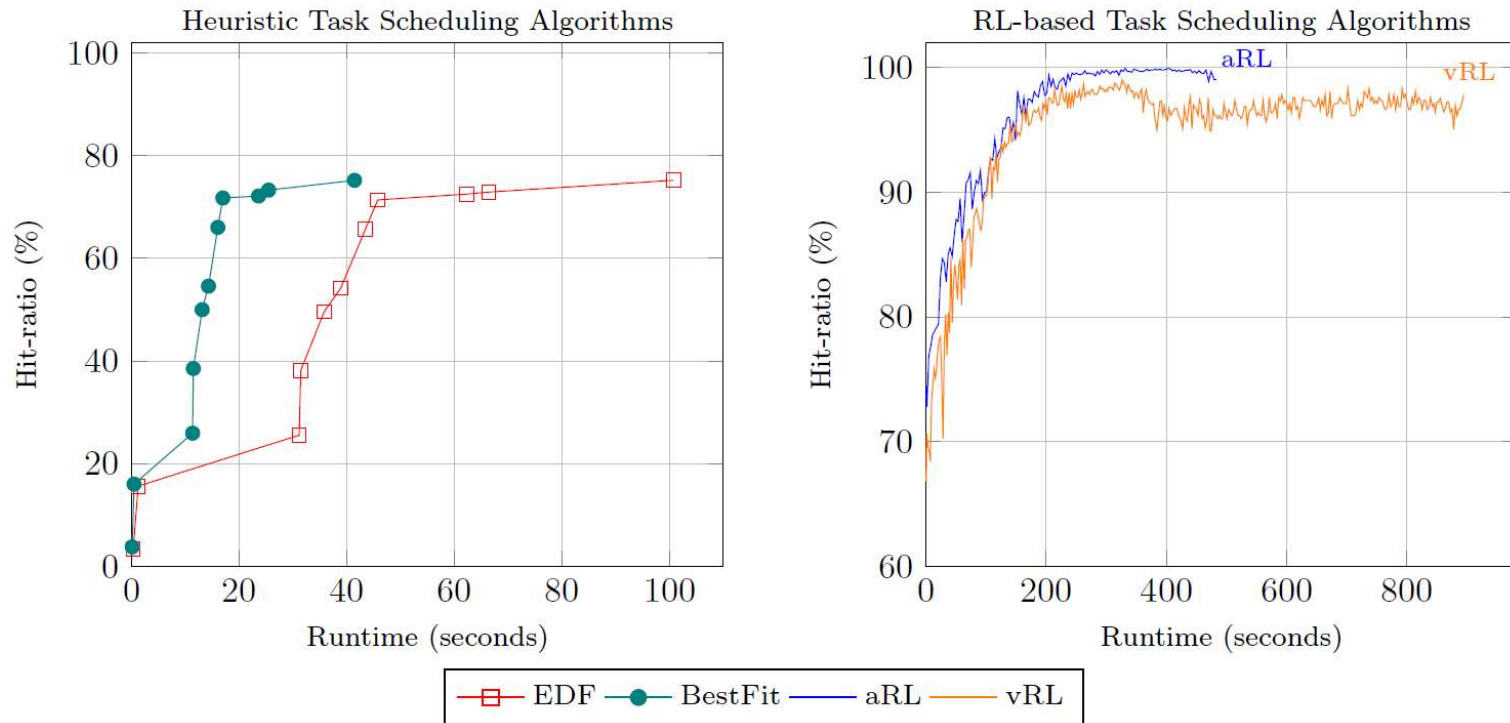


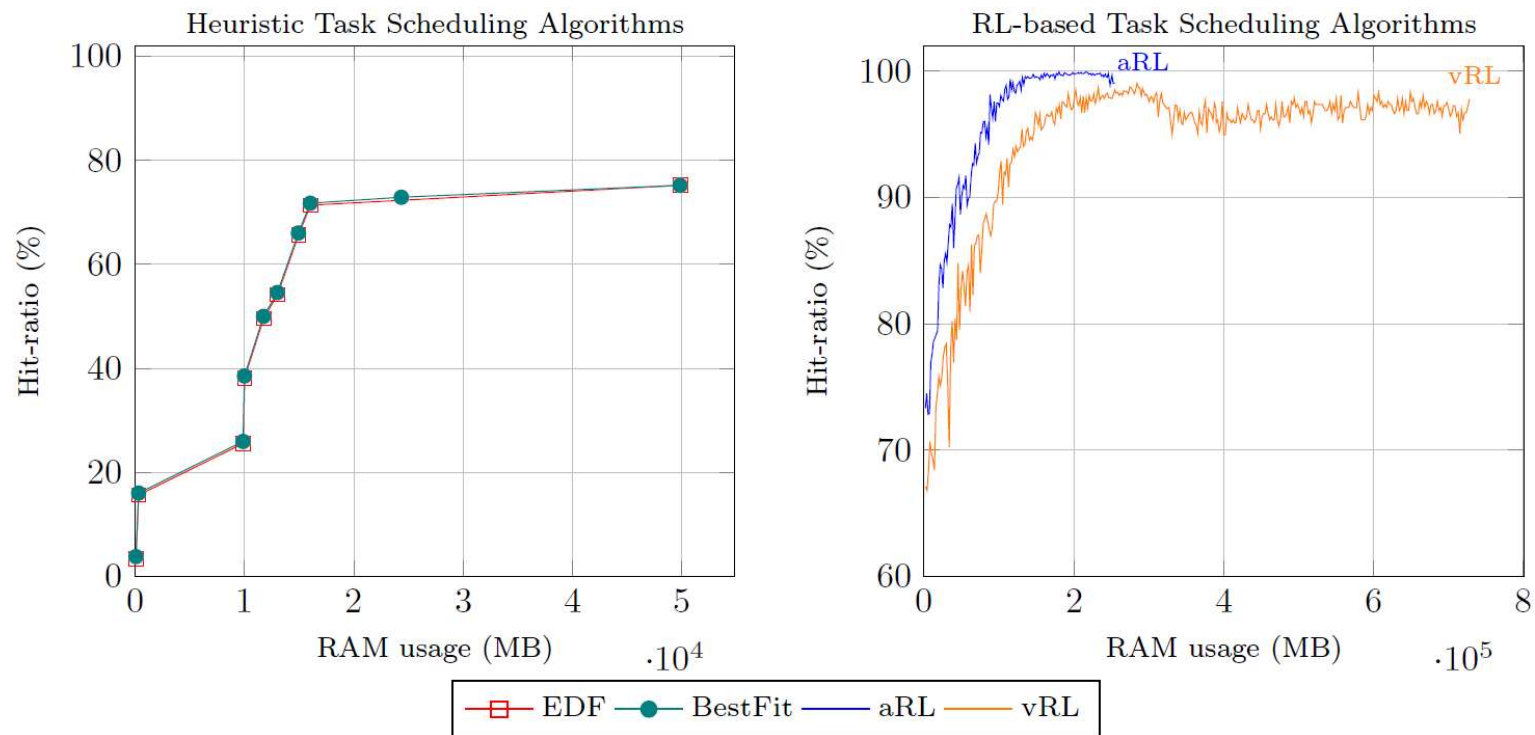
Fig. 2. Overview of the edge computing (EC) architecture.

## Experimental results - Runtime:



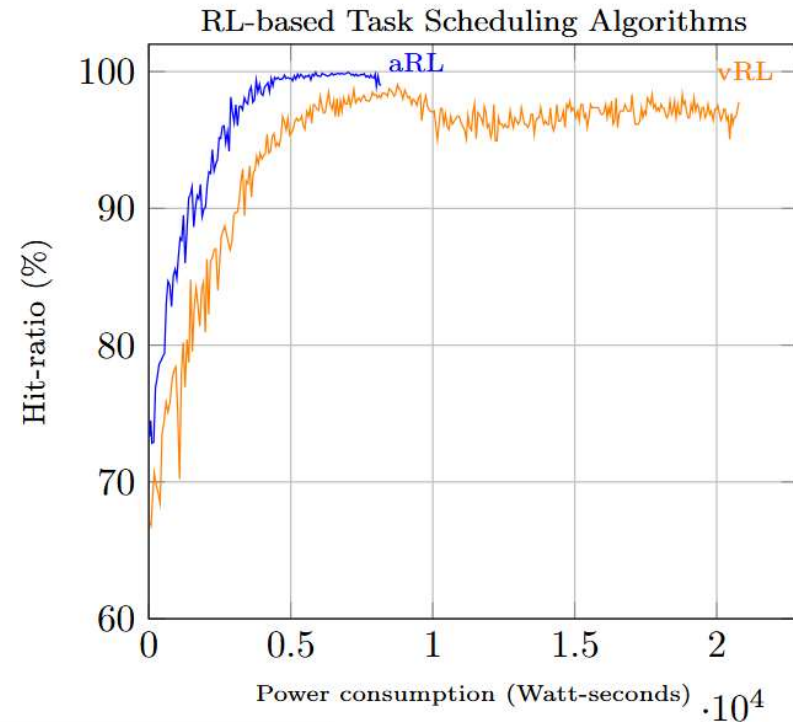
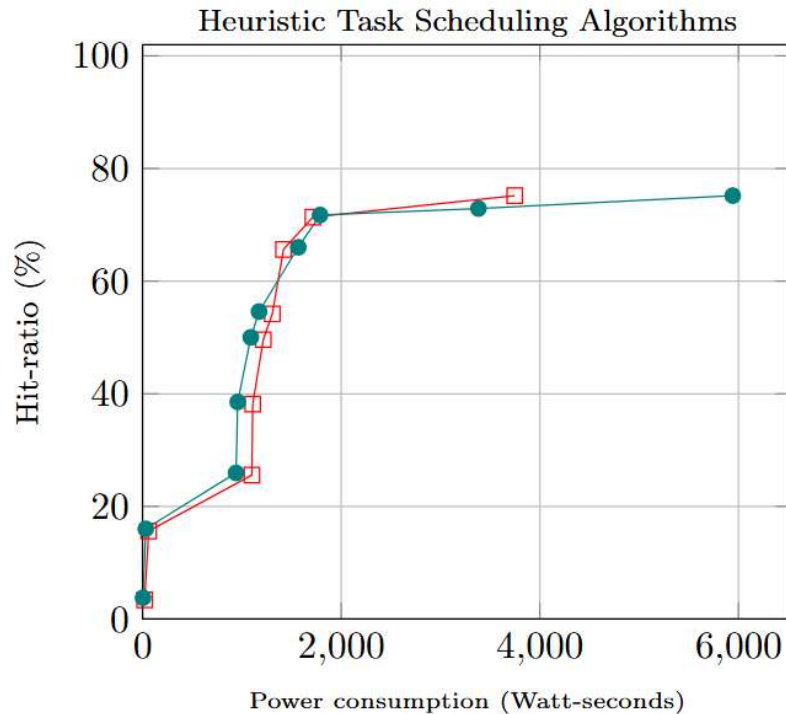
- aRL outperforms EDF, BestFit, vRL in hit-ratio by 24%, 24%, 1.9%.
- aRL is 46% faster than vRL.

## Experimental results – RAM usage:



- aRL meets hit-ratio target using 65.5% less RAM than vRL.
- EDF & BestFit fail to meet the target.

## Experimental results – Power consumption:



- aRL meets hit-ratio goal using 60.6% less power than vRL.
- EDF & BestFit fail to meet the goal.



## Conclusion:

- aRL schedules SRTAs in EC w/ reduced learning time (runtime).
- aRL aims to maximize hit-ratio w/ minimal runtime.
  - Achieved by incorporating 'informed exploration' & 'action masking'.
- aRL achieves highest hit-ratio among heuristic & RL baseline methods.
  - aRL reduces runtime (46%), RAM usage (65.5%), power consumption (60.6%) compared to vRL.